

Estimations: about to be solved

Data-backed, semi-deterministic software estimation on a delivery system — a method before the project, a living projection during it, a model that sharpens with every delivery.

Abstract. Software estimation has failed for fifty years for two reasons no statistics could fix: the delivery process itself was a random variable — every team a different machine — and the data describing it was self-reported fiction. rs-ip, the Rootstrap Delivery System, removes both by construction. For the first time there is a delivery system, not a methodology: an executable, versioned machine in which every project runs the same loop (plan, build, adversarial review, merge, QA convergence) against a human-validated Ground Truth, and every step leaves signed telemetry — run ledgers, provenance stamps, decision timestamps, QA bounce events. This paper presents the estimation method that substrate makes possible. Work is expressed in a stable unit, the **capability**, categorized ex ante into ten reference classes (tier C1–C5 × size S/L) whose priors converge to measured values by hierarchical Bayesian updating on the log scale. A Monte Carlo engine simulates the project over its real dependency graph under explicit metavariables — parallel tracks, per-person contextual multipliers, QA-dependent rework, project definition level, measured client decision latency — plus rare heavy-tailed shocks and shared frailties that correlate risk across capabilities. The deliverable is a probability distribution of dates (P50/P80/P95), reported as a shifted lognormal — whose CDF answers “what is the probability we are live by date X?” — while the engine remains empirical. A calibration-first validation protocol — coverage, PIT, CRPS, rolling-origin backtests against baselines — keeps the model honest. Under the reference configuration, the cold-start median for a C1-S capability is **one working day**; estimation becomes tractable when the process is standardized, the unit is stable, and delivery evidence continuously calibrates the forecast.

1 · Introduction: the industry had methodologies, never a system

Story points are dead. Not obsolete — **dead**. A relative, self-reported, team-local unit was the best the industry could do when the process changed on every project and nobody measured anything. Neither condition holds anymore. The only honest answer to “how long?” has always been a **statistical distribution** — everything else was a point estimate wearing a costume of confidence [2].

Estimation never failed for lack of mathematics. Function points, COCOMO, story points, planning poker — every generation rebuilt the same ritual and failed the same way, because what they had were **methodologies**: rituals for humans, locally reinterpreted by every team that adopted them. A methodology cannot be measured, because it is never the same machine twice. rs-ip is a different kind of object — a **delivery system**: an executable, versioned machine. Every project runs the same loop (plan, build, adversarial review, merge, QA convergence) by agents under human gates, against an exhaustive, human-validated **Ground Truth**; every capability is specced into user stories with acceptance criteria and evals before a line is built; and every step leaves evidence — signed run ledgers, git history, provenance stamps that version the very process that produced each unit of work, decision timestamps, QA bounce events. **The process became executable, observable, and versioned; the data became a sensor.** Those are precisely the conditions estimation always lacked.

The industry could not estimate because it could not measure. We measure.

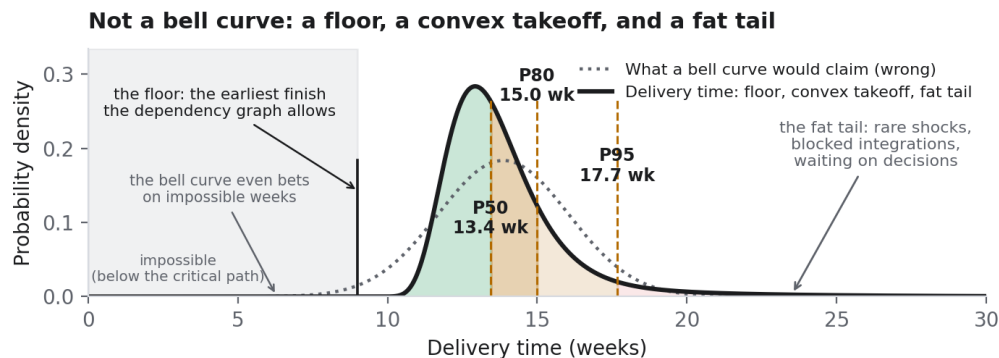


Figure 1 — The deliverable: a probability distribution over time, not a date. Density is exactly zero left of the critical-path floor — finishing faster than the graph allows is impossible (note the bell curve betting on those weeks). “P80: 15.0 weeks” reads: 8 projects in 10 like this one land on or before week 15.

Contributions. (i) A stable, checker-enforced unit of estimation — the capability — with an ex-ante frozen, ten-cell reference-class taxonomy (§2). (ii) A generative model of delivery with explicit, measured metavariables — parallelism, per-person multipliers, QA-dependent rework, definition level, client latency — plus heavy-tailed shocks and shared frailties for correlated risk (§3). (iii) A simulation engine over the live dependency graph that outputs empirical date distributions, criticality indices, and priced what-ifs (§4). (iv) A calibration-first validation protocol with proper scoring rules and baselines (§5). §6 describes operation: a one-day estimate before kickoff, a living projection after.

2 · The unit of estimation: capabilities, categorized ex ante

The delivery system already defines the unit. A **capability** is a business function — a verb, a unit of value you could scope, price and deliver on its own, composed of user stories with acceptance criteria and evals. Not a page, not a sprint. The definition is doctrine in the method, with litmus tests and a checker that rejects violations — which matters here for one reason: **a stable unit makes measurements comparable across projects**. Each capability is classified at Ground Truth time on two axes — **tier** C1–C5 (the nature of its complexity, a first-class field in the spec) and **size** S/L — at the same human gate that approves the spec. The label is **frozen ex ante**: actuals never rewrite it (§5).

	SIZE S — a few actions, one surface, one or two roles	SIZE L — many actions, several roles or surfaces
C1 · Single workflow known patterns, deterministic rules	C1-S · prior ~1 day <i>sign-up, roles & permissions, audit trail</i>	C1-L · prior ~2–3 days <i>admin CRUD over many entities, filters, exports</i>
C2 · Multi-workflow real business rules, failure states	C2-S · prior ~3–4 days <i>subscription checkout with dunning states</i>	C2-L · prior ~1 wk <i>support ticketing: SLA timers, two views</i>
C3 · Cross-functional external systems we do not control	C3-S · prior ~1 wk <i>intake + eligibility check against a 3rd-party API</i>	C3-L · prior ~2 wk <i>order-to-invoice with ERP sync + finance views</i>
C4 · Platform / probabilistic agentic, heavy eval surface	C4-S · prior ~1–2 wk <i>document ingestion with confidence thresholds + evals</i>	C4-L · prior ~2–3 wk <i>agentic triage: tool contracts, human gates, evals</i>
C5 · Strategic irreversible, regulated, novel	C5-S · prior ~3 wk <i>payments/compliance core, migration cutover</i>	SLICE IT — a C5-L is not a capability; it is several. Anything past ~4 weeks is a scoping error, not an estimate.

Table 1 — The ten reference classes, with cold-start priors. **Reference configuration for every prior:** one delivery track ($W = 1$), a median operator ($m_p = 1$), QA per tier policy, fully validated spec ($g(D) = 1$), no open decisions ($L = 0$), no shocks ($X = 0$); durations are medians of elapsed working time, builder dispatch → QA convergence, after the Ground Truth is approved (foundations and launch excluded). Every departure from this configuration is handled by the model, not by the table.

Under the reference configuration, the cold-start median for a C1-S is ONE WORKING DAY — builder dispatch to QA convergence. The ladder no longer measures typing; it measures what agents compress least: external systems, eval convergence, and waiting for humans.

The numbers are small because delivery is agentic: the system, not a person, does the building, and it knows its own method. What remains expensive climbs the ladder — third-party integration (C3), probabilistic surfaces that need eval convergence (C4), and irreversible decisions with human gates (C5). Read the cells as **reference classes** [3], not promises: the prior is each class's cold start, treated as tau pseudo-observations; from the first delivery on, a cell is the measured history of similar work, and new work is projected against what it resembles — agentic against agentic, CRUD against CRUD. Ten cells, not a hundred-point scale, because categorical judgments against a validated Ground Truth are where human classification is reliable. When an actual lands far outside its band, the miss is recorded as a *miss*; any reclassification is a new ex-post label for future work, never a rewrite of the ex-ante one (§5).

What estimation requires — the complete list

1 · Exhaustive documentation → a Ground Truth. PRDs, manuals, transcripts, an existing codebase — whatever exists. The intake mines all of it into a validated model of the business (capabilities, stories, decisions, dependencies). Clients using AI usually have this; then this step takes hours.

2 · The bill of capabilities. Every capability categorized (tier × size) at the approval gate — plus the dependency graph the Ground Truth already carries (machine-checked *depends_on*).

3 · The team configuration. Parallel tracks (developers/operators), which people, QA policy — plus the Ground Truth's own completeness grade, measured, not asked. These set the metavariables of §3.

That is the whole input. Documents in the morning; a validated Ground Truth, its categorized capability bill, and the delivery distribution — Figure 1 — by end of day. **One day.** Done on live engagements: client documents to a published, client-facing Ground Truth in a single working day.

Where every number comes from. Nothing in this paper asks anyone to fill a form. Each model term maps to a mechanism the delivery system already runs:

Model term	The rs-ip mechanism that measures it
Duration actuals	Delivery records: builder dispatch → PR merged → QA-converged, timestamped in runfiles and story <i>delivery</i> : entries
Rework rate r	QA-convergence bounce events — first-class entries in the outcome ledger
Client latency L	DECIDE / VALIDATE timestamps in run ledgers: every decision, raised → resolved
Definition level D	Ground Truth health counts: approved vs draft artifacts, open decisions per capability
Multiplier m_p	Signed, attributable runs — every run names its agent and its human
Process version	Provenance stamps (<i>agent@version · via loop@version</i>) — segment and recency-weight by the process that did the work
Category (frozen)	Tier × size in spec frontmatter, set at the same human gate that approves the spec

Table 2 — Provenance of every model input. The estimation model is a read-out of the delivery system, not an add-on process.

The estimate is a product in itself. A validated model of your business plus a defensible, data-backed delivery distribution has stand-alone value — for capital conversations, roadmap decisions, build-vs-buy calls — whether or not the build starts tomorrow. And because the system is stateless and installs by convention, this **scales**: any team, any project, same method, same priors, one shared ledger — every engagement inherits the calibration of all the others.

3 · The generative model: durations, metavariables, learning

Base durations — the working family. Each capability's duration is a random variable, never a number. Delivery time is floored on the left, right-skewed, with a real tail — not Gaussian. Our working family for the base build time is Weibull [6], standard for time-to-completion and interpretable through its hazard (the chance of finishing now, given not finished yet); seed shapes k between 1.4 and 1.9 are initial priors that must re-earn their fit under the §5 protocol — like everything else in this system, the distribution keeps its seat by winning it. For capability i in cell $c(i)$, staffed by person $p(i)$:

$$B_i \sim \text{Weibull}(k_{c(i)}, \lambda_{c(i)} \cdot g(D_i)) \cdot m_{p(i)}, \quad k_c > 1 \quad (1)$$

A model choice, stated as one: **Weibull is a working family, not a law of the system** — §5 makes it compete out-of-sample against lognormal, gamma, mixtures and the empirical bootstrap, and the engine itself is distribution-agnostic (§4).

The metavariables. Everything in (1)–(4) beyond the cell's own parameters is a metavariable — a property of the team and the project, not of the work. They are few, named, and **measured, not asked** (Table 2):

W — parallel tracks, and $c(\cdot)$ — coordination, counted once. The scheduler runs at most W capabilities in flight and already captures the *structural* loss of parallelism — dependencies force idle tracks. Coordination loss therefore enters exactly once more, as a measured inflation $c(\cdot)$ of in-flight durations under concurrency, calibrated jointly with the scheduler so nothing is penalized twice. The familiar sublinear-throughput summary (effective parallelism close to W to the alpha, alpha 0.7–0.9) is the back-of-envelope consequence [1], not an extra assumption in the engine. The dependency graph's critical path remains the hard floor no headcount can cross — the model states, in weeks, when the next track buys nothing.

m_p — the contextual delivery multiplier. Every run is signed, so each person's actuals against their cell's median are measurable; the ratio, shrunk toward 1.0 exactly as in (5), scales the base time. Two disciplines keep it honest. First, it is a **contextual** multiplier — partially confounded with task mix, client, and definition quality — so it is estimated conditionally on tier and D, reported with its credible interval, and never marketed as a performance score. Second, its spread is **narrow by construction**: in agentic delivery the system does the building, so the individual variance that dominated human-era estimates is largely absorbed — the operator matters less than the developer of old, and what m_p prices is judgment (spec quality, gate latency, review sharpness), not typing speed.

Supporting roles are metavariables too — with second-order effects. Project management, configuration management/DevOps, design: each has a measurable channel — PM mostly moves L (decisions chased sooner) and D (cleaner definition); DevOps moves the bounce clock (deploy freshness, re-test turnaround inside r); design moves rework on UI-bearing capabilities. All are estimated from the same ledger, and all are **expected to be second-order** next to the developer/operator terms under the reference delivery loop — a hypothesis the ledger tests, not a decree: on a given project DevOps or decision management can become the binding constraint, and the model will say so.

r — rework, and how QA is modeled. QA does not change the first pass; it changes the **re-entries**. Every story bounces through QA convergence in the loop, each bounce a ledger event, so rework is measured, not assumed: re-entry counts are geometric with rate r, conditioned on tier and QA staffing. And rework is **cheap in this system**: a bounce is a Ground Truth correction plus an agent re-run — not weeks of human re-typing — so re-entry costs are small fractions of base time. QA's effect is reported as a **calibrated association**, conditioned on tier and definition (risky work attracts QA — the confounding is stated, not hidden): dedicated QA lowers r and catches defects at story level rather than integration, truncating exactly the deep re-entries in the tail. It moves the P95 far more than the P50 — in staffing terms, C1 is agent-verified end-to-end, mid tiers take half-time QA, top tiers half to full.

D — project definition level. The Ground Truth grades itself, per capability: approved vs draft artifacts, open decisions, unresolved questions — the Explorer's own health counts. D scales base time through $g(D) \geq 1$ and widens the cell (lower k) when definition is poor. This is rolling-wave planning made formal: near, validated capabilities enter tight; far, still-draft ones enter wide — and the model names which validations narrow the curve most.

L — client decision latency. Run ledgers timestamp every decision from raised to resolved, so each client's responsiveness is a measured distribution, sampled onto capabilities carrying open decisions. The client is *in* the model — and can see what their own latency does to their date.

X and shared frailties — rare shocks, correlated risk. Weibull tails are real but not heavy; exceptional events — an architecture rethink, an indefinitely blocked integration, a regulatory surprise — need their own term: X is a rare, heavy-tailed mixture component (2). And because real delays arrive *together* — one vague definition touches five capabilities, one dead API blocks a set — trials draw **shared frailties** [8]: project-level factors (scope, integration, staffing, client slowdown), sampled once per trial and applied multiplicatively to every capability they touch (3). Uncertainty also lives in the **graph itself**: with tier- and definition-dependent probability, a trial activates **hidden dependencies** — optional edges standing for the integration nobody knew about yet — so discovery changes the topology a trial schedules, not just a duration, and forecasts are stress-tested against alternative graphs. Without correlation and sampled topology, Monte Carlo produces a curve that is too narrow — precision that is a lie:

$$d_i = B_i + \sum_{j=1}^{N_i} R_{ij} + L_i + X_i, \quad N_i \sim \text{Geom}(r_c(Q)), \quad X_i \sim \pi_c \mathcal{H} \quad (2)$$

$$\tilde{d}_i = d_i \cdot \exp\left(\sum_f a_{if} u_f\right), \quad u_f \sim \mathcal{N}(0, \sigma_f^2) \quad (\text{shared frailties} = \text{correlated risk}) \quad (3)$$

$$T^{(t)} = \text{DAGschedule}(\{\tilde{d}_i^{(t)}\}, W, c(\cdot)) \quad (\text{coordination counted once, in } c) \quad (4)$$

A Monte Carlo without correlated shocks is a lie of precision. Real delays arrive together — the shared frailties are what keep the curve honest.

Learning — partial pooling on the log scale, and drift. Durations are log-scale objects: each cell starts at its prior treated as tau pseudo-observations, and every delivered capability contributes its telemetry-reconstructed duration, giving the precision-weighted blend (5) — empirical-Bayes shrinkage [4], hierarchical across cells so small cells borrow strength from their tier. The simulation draws from **posterior predictive** distributions, so parameter uncertainty flows into the forecast

instead of being plugged in as a point. Because the delivery system itself improves, the process is **non-stationary by design**: every run carries its loop version in the provenance stamp, so fits are segmented by process version, recency-weighted (exponential decay), and screened for structural breaks — old data informs, recent data governs.

$$\log \hat{\mu}_c^{\text{post}} = \frac{n_c \overline{\log x_c} + \tau \log \mu_c^{\text{prior}}}{n_c + \tau} \quad (\text{durations are log-scale objects}) \quad (5)$$

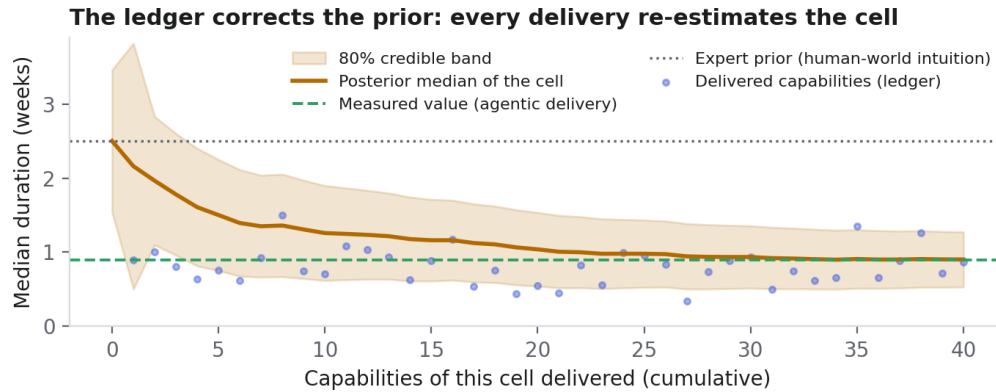


Figure 2 — The ledger corrects the prior. Here an expert prior of 2.5 weeks — human-world intuition — converges onto the measured agentic value of 0.9 weeks as deliveries accumulate. One project contributes ten to thirty observations, because the unit of observation is the capability. More telemetry bandwidth in, more precision out.

4 · The simulation: Monte Carlo over the real dependency graph

Why simulation, not a formula. A project composes skewed distributions over a DAG under resource limits, rework loops, and shared shocks. That composition has no closed form — and Monte Carlo [7] needs none: instead of solving the project, **play it**. Ten thousand times.

Algorithm 1 — one trial of the delivery simulation (repeat 10,000×):

- 0 · Sample the project.** Draw the shared frailties (3) — scope, integration, staffing, client slowdown — once for this trial, and activate hidden dependencies with tier- and definition-dependent probability: this trial's DAG is sampled, not assumed. This is what correlates risk.
- 1 · Sample the work.** For every remaining capability, draw base time B_i from its cell's *posterior predictive* (1), scaled by the multiplier of whoever is staffed on it.
- 2 · Sample the rework.** Draw QA re-entries N_i (rate r from tier + QA policy). In rs-ip a bounce is a Ground Truth correction plus an agent re-run — frequent but small.
- 3 · Sample the waiting.** For every capability carrying an open decision, draw client latency L_i from that client's measured distribution.
- 4 · Sample the exceptional.** With small, tier-dependent probability, add a heavy-tailed shock X_i (2) — the architecture rethink, the blocked integration.
- 5 · Schedule the graph (4).** List scheduling: whenever a track frees, start the highest-priority READY capability — dependencies shipped, P0 before P1, the same ready rule the delivery loop runs daily — at most W tracks, concurrency inflation $c(\cdot)$ applied once.
- 6 · Record the finish week T .** Sort the 10,000 outcomes: P50 is the 5,000th value, P80 the 8,000th, P95 the 9,500th — quantiles read directly off the empirical distribution.

What the output is — and how it is reported. The output is the distribution of 10,000 played projects, and it has a characteristic shape the simulation produces by itself: a **hard floor at the critical path** (no trial can finish before the graph allows — density is exactly zero there), a **fast, convex rise to a well-defined peak**, and a **slower fall into a long right tail**. The client-facing form is a **shifted lognormal** — right-skewed by nature, location at the floor — whose CDF answers the question clients actually ask: “*what is the probability we are live by date X ?*” The fit is used **only when it preserves the simulated quantiles and tail**; otherwise the empirical curve is reported directly — if the picture and the data ever disagree, the data wins. And the regime is stated openly: **cold start is parametric** — the same simulation run over the priors — and the empirical engine takes over as the ledger fills with this project's own deliveries.

The engine is empirical. The shifted lognormal is the language we give the client — a picture whose CDF answers “will we be live by June 1?” Time is an output we measure, never an input we invent.

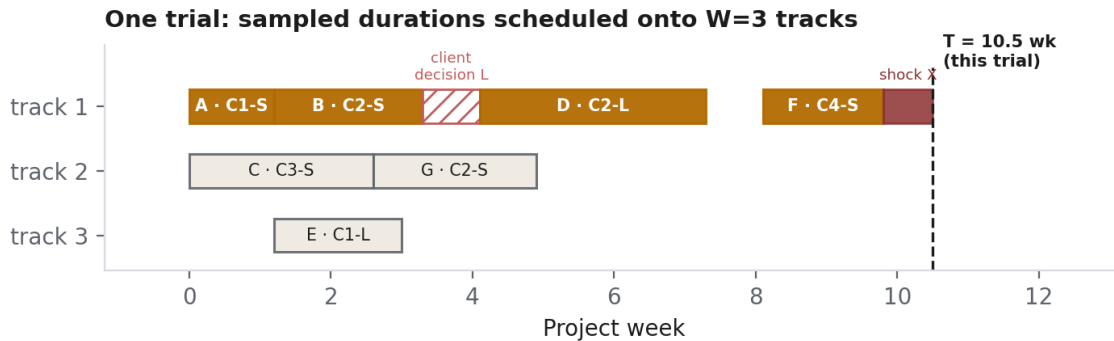


Figure 3 — One trial of the scheduler: sampled durations on W=3 tracks in dependency order. The gold chain set this trial's finish date; the hatched gap is a sampled client-decision latency; the dark segment is a rare heavy-tailed shock X landing on the critical path.

The critical path is an output, not an opinion. Each trial has a longest chain (gold, above). Across 10,000 trials every capability earns a **criticality index** — the share of trials in which it sat on that chain. Sorting by it yields *the order of things*: which capabilities to start first, which decisions to chase this week, which single validation buys the most narrowing. And one clarification that keeps everyone honest: **the date never decides the order**. Value and dependencies order the backlog — the client's call, on the client's priorities; the simulation then checks whether the value-ordered plan fits the date. When it doesn't: slice finer, parallelize, or renegotiate scope — in that order, never by massaging the number.

Simulation prices questions. Add a track, re-run, watch the curve move — or not, if the critical path binds. Add dedicated QA and watch the P95 pull in while the P50 barely moves. Every staffing debate becomes thirty seconds of arithmetic on the real project graph.

5 · Validation: calibration is the product

A forecast that cannot be graded is an opinion with a chart. The protocol that keeps this model honest has four parts, and every part runs on the ledger the system already writes.

Ex-ante freeze. Category, definition level, team configuration and the forecast itself are recorded at estimate time — they are Ground Truth artifacts and ledger entries anyway. Actuals never relabel the past: a miss stays a miss on the record, because retrospective reclassification silently converts prediction errors into labeling errors and makes any model look better than it was. Calibration is always evaluated against the label chosen *before* the outcome was known.

Coverage, sharpness, and proper scores. Of everything called P50, about half must land on or before; of everything called P80, about 80%; P95, about 95% — coverage first, verified with probability-integral-transform (PIT) checks. Subject to calibration, sharper is better [5]: interval width (P50–P95) is the second metric, and full-distribution accuracy is scored with CRPS and pinball (quantile) loss — proper scoring rules that cannot be gamed by hedging. Evaluation is **rolling-origin**: fit on the past, forecast the future, roll forward — never scored in-sample.

Scope-normalized calibration vs current-plan projection. Scope moves — that is a feature, not an error — so the ledger keeps two series. The **frozen-scope forecast** (the bill of capabilities as it stood at estimate time) is what accuracy is graded on; the **current-plan projection** (approved changes included) is what the project is managed on. Without the split, added scope masquerades as model error — or a moving baseline quietly flatters the model. A change order re-baselines the plan, never the grade.

Two report cards, two clocks. Capability-level calibration converges fast — every project contributes ten to thirty cell outcomes plus rework, latency and shock telemetry, so the base distributions sharpen in months. Project-level calibration (the P50/P80/P95 of full completion dates) accrues one observation per project and is stated for what it is: the slower card, published with its sample size, convincing on the timescale of a portfolio rather than a quarter.

Baselines and the family tournament. The model must beat what is free: the human point estimate, and a naive throughput extrapolation. And the reporting family must win its seat: the shifted lognormal competes out-of-sample against Weibull, gamma, mixtures, and the empirical bootstrap — whichever preserves quantiles and tail behavior best is the one the client

sees. The product is committed to the engine and the protocol, never to a distribution.

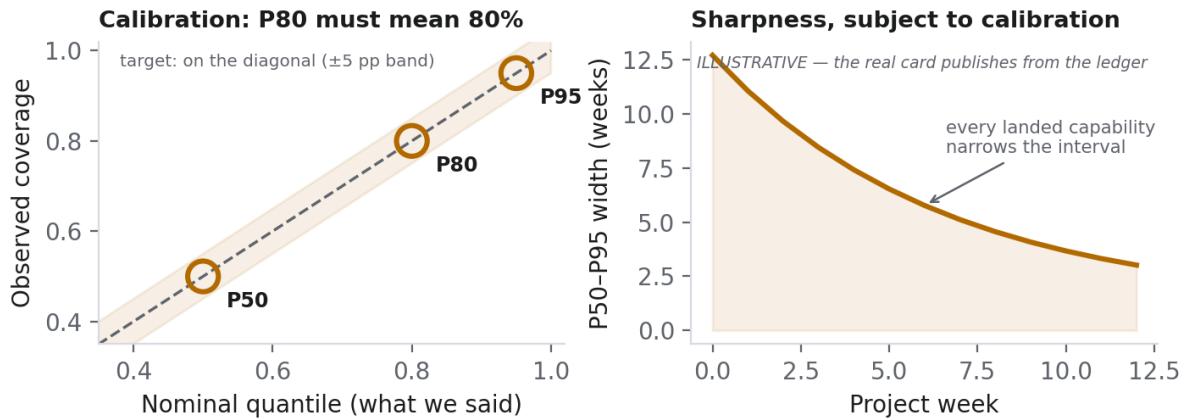


Figure 4 — The report card (illustrative). Left: coverage calibration — every stated quantile must land on the diagonal within tolerance. Right: sharpness earned over the project as landed capabilities replace samples. The real card publishes from the delivery ledger.

Calibration is the product: P80 must come to mean 80%. The report card publishes from the ledger — not before, and not from anyone's memory.

6 · Operating it: one day to estimate, a living curve after kickoff

Before the project — the method. Ingest the client's documents into a Ground Truth; validate it with the operator; categorize every capability at the approval gate; declare the team. Then the estimate is mechanical: bill of capabilities + dependency graph + team configuration in, Figure 1 out. Nobody in the room is guessing — the priors carry every previous project, and the client sees the assumptions (the categorized list itself), not a number pulled from experience. **The first categorization is the act that matters most** — the category is the covariate everything conditions on — so it is treated like approval: a named human act on a validated spec, frozen ex ante, checked by gates. (Foundations — the Ground Truth work itself — and launch are short, near-fixed phases bookending the simulated window.)

After kickoff — the living projection. Delivered capabilities stop being samples and become facts; remaining ones are re-sampled from posteriors this very project keeps sharpening. The simulation re-runs continuously; the fan narrows; slip appears as a probability shift weeks before any status meeting would admit it — in the same Explorer where the client already reads the capability map and decisions. Partial information flows in as it arrives: answered questions, landed stories, a decision recorded mid-call.

Partial answers are information. It is a stream — UDP, not TCP.

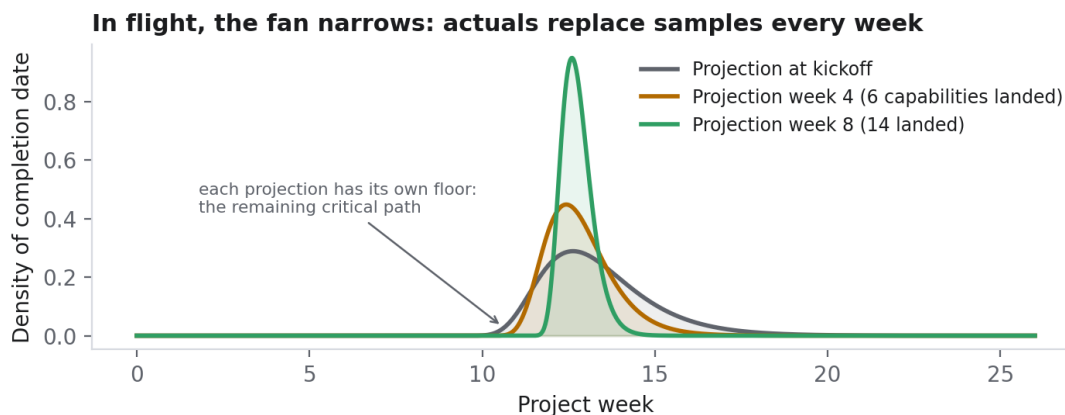


Figure 5 — The same project at kickoff, week 4 and week 8. Certainty is earned by delivery, not asserted by a plan.

How to communicate it. Three numbers, one sentence each: **P50** — “the coin-flip date; we plan to it.” **P80** — “the date we commit to; 8 in 10 land on or before.” **P95** — “the date we price risk to; overrunning it is a 1-in-20 event.” And any hard date is

one lookup: the curve's CDF at that date *is* the probability of making it. Never hand over a single date again; hand over the curve and read it aloud. Scope changes stop being arguments: a new ask is a new capability, categorized in minutes, its date and price impact simulated before anyone commits. And the client's own decision latency is in the model — showing them how their responsiveness moves their P80 turns consulting's oldest passive-aggressive conversation into a graph.

Pricing follows the categories. A capability is a unit of business value — which makes it a unit of *purchase*. Each of the ten cells carries a price band: **price per capability, from the same categorization rs-ip already performs**. A proposal is the priced bill of capabilities; a change order is one more row. Incentives finally align: hourly billing put both sides on opposite ends of a clock; per-capability pricing puts both sides on outcomes. And on top, **fixed price with stated confidence**: commit the engagement at the P95 range — a boundary with a model-implied 5% residual, priced as an actuarial premium from the P50→P95 spread rather than a padding habit, and hardening into a *calibrated* premium as the \$5 report card matures.

The claim, made carefully — and boldly. Estimation becomes tractable when the delivery process is standardized, the work unit is stable, and actual delivery evidence continuously calibrates the forecast. Those three conditions were never available together before; on a delivery system they hold by construction. Novel C5 work keeps irreducible uncertainty and the model says so out loud — that is what a wide P50–P95 gap is for. But for the large majority of software projects — compositions of C1–C4 capabilities this system has delivered many times — estimation is about to stop being a judgment problem at all. The process is fixed. The telemetry is flowing. The priors are converging on a schedule of months, not decades, and the delivery system keeps removing variance at the source. **Semi-deterministic, data-backed estimation is not a research agenda; it is an engineering consequence already in motion.** “How long will it take?” is about to have, for the first time in this industry's history, an answer that is not a guess: a curve, backed by every project before yours, sharper every week.

Limitations. The model inherits the quality of its taxonomy and its Ground Truth: a miscategorized bill or a thin GT widens honestly (through D) but cannot be fully repaired downstream. Cold-start cells lean on priors until the ledger fills them. Person and QA effects are observational — conditioned and interval-reported, never causal claims. Truly novel C5 work keeps irreducible uncertainty, and the calibration report card — not this paper — is the evidence that the promises above hold.

References. [1] F. P. Brooks, *The Mythical Man-Month*, 1975. [2] D. Kahneman & A. Tversky, “Intuitive prediction: biases and corrective procedures,” 1979. [3] B. Flyvbjerg, “From Nobel Prize to project management: getting risks right,” *Project Mgmt. Journal*, 2006. [4] B. Efron & C. Morris, “Stein's paradox in statistics,” *Scientific American*, 1977. [5] T. Gneiting & A. E. Raftery, “Strictly proper scoring rules, prediction, and estimation,” *JASA*, 2007. [6] W. Weibull, “A statistical distribution function of wide applicability,” *J. Applied Mechanics*, 1951. [7] N. Metropolis & S. Ulam, “The Monte Carlo method,” *JASA*, 1949. [8] J. Vaupel, K. Manton & E. Stallard, “The impact of heterogeneity in individual frailty on the dynamics of mortality,” *Demography*, 1979.

Rootstrap · Delivery System — August 2026. Priors in Table 1 and the seed values of $c(\cdot)$, r , $g(D)$ and the frailty variances are illustrative; live values are maintained in the delivery ledger, versioned by loop, and updated with every merged capability. The engine is empirical; any reporting distribution must re-earn its seat under the \$5 protocol on every run.